

University of Exeter

Department of Computer Science

ECM3412

Programming Task and Report

Candidate number: [740007415]

Abstract

This report evaluates Ant Colony Optimisation (ACO) for a fixed-bin load-balancing problem. Each ant assigns every item to one bin and fitness is the difference between the heaviest and lightest bin. Two prescribed instances were tested with population sizes $p \in \{10, 50\}$ and evaporation rates $e \in \{0.05, 0.30\}$. Each setting used five independent trials and 10,000 fitness evaluations. BPP1 favoured $p = 10, e = 0.30$ (mean final fitness 1091.60), whereas BPP2 favoured $p = 50, e = 0.30$ (mean 143855.55). An additional $p = 25, e = 0.15$ experiment did not improve either result. High evaporation was beneficial on both instances, but the best population size depended on the problem.

1 Introduction

The classical bin-packing problem assigns weighted items to capacity-constrained bins while minimising the number of bins. This project studies a related fixed-bin balancing problem. Every item must be assigned to one of a prescribed number of bins, and the objective is

$$d = \max_j L_j - \min_j L_j,$$

where L_j is the load of bin j. Lower d is better, with zero representing perfect balance.

The project implements ACO using a construction graph whose rows represent items and columns represent bins. Ants select bins according to pheromone values; each route deposits $100/(d + 1)$ on its selected edges, all pheromone is multiplied by $(1 - e)$, and a trial stops after 10,000 fitness evaluations. BPP1 contains 600 items and 12 bins with $w(i) = 2i + 1$; BPP2 contains 400 items and 40 bins with $w(i) = i^2/4$. The experiments investigate how population size and evaporation affect performance.

2 Literature Review

Bin packing is a family of combinatorial optimisation problems with objectives including bin count, unused capacity, and load balance. Classical one-dimensional bin packing is NP-hard, making heuristic and metaheuristic methods useful for large instances [9, 5]. A recent survey of one-dimensional bin-packing research also shows that population-based and hybrid metaheuristics remain active areas of study [8]. However, most published studies minimise the number of capacity-constrained bins, whereas this project fixes the number of bins and minimises load range. This difference is important when comparing algorithms.

2.1 Ant Colony Optimisation

ACO is a constructive metaheuristic inspired by pheromone-mediated ant foraging. Artificial ants probabilistically construct solutions, good routes deposit pheromone, and evaporation prevents all future search from following old decisions [4, 3]. Positive feedback supports exploitation, while probabilistic construction provides exploration.

In this project, pheromone entry τ_{ij} represents assigning item i to bin j. An ant makes one decision for each item, so a complete route is a complete assignment. The implementation uses no local heuristic: the probability of choosing a bin is proportional only to the corresponding pheromone row. After calculating d, every used edge receives $100/(d+1)$ and the whole graph is evaporated. This directly follows the brief and makes the effects of p and e experimentally visible.

Levine and Ducatelle apply ACO to bin-packing and cutting-stock problems. They report that the "pure ACO approach can compete with existing evolutionary methods" and that adding local search can improve performance for some problem classes [6]. Their work supports ACO's suitability for discrete packing, but also shows that representation and local improvement matter. More sophisticated variants such as MAX–MIN Ant System restrict pheromone values and update rules to reduce stagnation [10]; those additions are outside this coursework implementation.

2.2 Genetic Algorithm

Genetic Algorithms use selection, crossover, and mutation to evolve a population. A bin-packing chromosome may encode direct item-to-bin assignments, item permutations, or groups of items. Generic crossover can destroy useful groups or create poor assignments, so effective methods require problem-specific operators or repair.

Yakawa, Iima and Sannomiya design crossover to inherit item combinations within bins and mutation to encourage capacity-feasible packing. They aim to generate "diverse offsprings" and report frequent optimal solutions on their benchmarks [11]. Chan *et al.* likewise show that a GA can incorporate quality-related objectives rather than treating bin count as the only measure of solution quality [2]. A GA could represent this project's solution directly, but its crossover would need to preserve balanced groups rather than only feasibility. Compared with ACO, a GA explicitly retains competing complete solutions; its disadvantages are operator design and possible repair overhead. A fair comparison would also require the same fitness-evaluation budget.

2.3 Particle Swarm Optimisation

PSO models social information sharing between particles. Standard PSO is continuous, whereas a bin assignment is a discrete categorical decision. A packing PSO therefore needs a discrete representation, a rule for assignment changes, and often repair or local search.

Liu *et al.* propose an evolutionary PSO for a two-dimensional multiobjective bin-packing problem involving bin usage and load balance. Their Pareto-based method

is relevant because it treats balance explicitly rather than only minimising bin count [7]. PSO may spread good assignments rapidly through a population, but its velocity concept is not naturally defined for item-to-bin choices. It is therefore a plausible alternative, but its implementation would require more problem-specific design than the ACO used here.

2.4 Other Nature-Inspired Approaches

Simulated Annealing (SA) is a physics-inspired method based on controlled cooling. It accepts some worsening moves early in the search and becomes more selective as temperature decreases. Rao and Iyengar report significant improvement over the heuristics they tested on a bin-packing variant [9]. For this project, an SA neighbourhood could move or swap items between bins and directly evaluate the change in d. Its results would depend heavily on the cooling schedule and neighbourhood.

Hybrid methods combine a population or stochastic search with domain-specific improvement. Alvim *et al.* combine lower bounds, load redistribution, and tabu search for one-dimensional bin packing [1]. Hemmelmayr, Hirsch and Schmid combine variable neighbourhood search with lower bounds and dynamic programming for variable-sized bin packing [5]. These are not pure nature-inspired algorithms, but they are important comparators because they show that a well-designed neighbourhood can be as important as the high-level metaphor. The recent survey by Munien and Ezugwu similarly identifies hybridisation as an important direction for improving bin-packing metaheuristics [8].

2.5 Critical Analysis

The published algorithms are not directly comparable because they often use different objectives, constraints, dimensions, and benchmarks. A method that minimises the number of capacity-constrained bins may not be best for a fixed number of bins and a load-range objective. Table 1 compares the main approaches specifically against the problem representation used in this project.

ACO is the most direct fit for the coursework because every item-to-bin assignment is a clear construction decision and pheromone can be attached directly to that decision. It is also transparent: the probability rule, reward, evaporation rate, and evaluation budget can be stated and tested explicitly. The required implementation is intentionally basic; it has no local search, pheromone bounds, heuristic information, or elitist global-best retention. These omissions make it less sophisticated than several published hybrids but provide a controlled experiment on population size and evaporation.

The literature therefore exposes a specific gap addressed by this study. Published work demonstrates that ACO, evolutionary algorithms, PSO, and hybrid methods can perform well on several bin-packing variants, but those results generally concern minimum-bin or multidimensional objectives. This project instead holds the number of bins fixed and evaluates load-range minimisation under an identical 10,000-

Table 1: Critical comparison of approaches for the fixed-bin balancing problem.

Approach	Representation	Strength for this project	Limitation
ACO	Item–bin pheromone graph	Directly represents each discrete assignment and learns from complete routes.	Positive feedback can cause stagnation; performance depends on evaporation and update policy.
GA	Assignment, permutation, or grouping chromosome	Crossover can preserve useful item groups and maintain several complete solutions.	Generic crossover may destroy balance; repair and grouping operators are difficult to design.
PSO	Discrete assignment particles	Good solutions can spread rapidly through social information sharing.	Continuous velocity has no natural meaning for categorical bin choices.
SA / hybrid search	Item moves, swaps, and neighbourhoods	Local moves directly target the load-range objective.	Cooling, neighbourhood design, and local-search cost strongly affect performance.

evaluation budget. Its contribution is consequently not a new metaheuristic, but a controlled empirical analysis of how population size and evaporation affect a basic ACO on two structured weight sequences. A hybrid ACO, grouping GA, or discrete PSO might perform better, but that claim would require a new implementation and equal evaluation budget.

3 Description of Results

3.1 Experimental Setup

The construction graph contains one row per item and one column per bin. Pheromone values are initialised randomly in $[0, 1)$. Roulette-wheel selection chooses each bin with probability proportional to its pheromone value. For each population, routes are evaluated, deposits are applied, and evaporation follows. The required settings were $(p, e) = (50, 0.05), (50, 0.30), (10, 0.05), (10, 0.30)$. Each setting was run five times on BPP1 and five times on BPP2, producing 40 trials. Every trial used a distinct seed and exactly 10,000 evaluations. Results below use the best ant in the final population, as required by the brief.

Table 2: Required trials. Mean, standard deviation, best and worst final fitness; lower is better.

Problem	p	e	Mean	SD	Best	Worst
BPP1	50	0.05	8163.00	654.19	7550.00	9138.00
BPP1	50	0.30	6344.00	768.19	5227.00	7073.00
BPP1	10	0.05	9141.40	1763.18	7005.00	11319.00
BPP1	10	0.30	1091.60	253.82	778.00	1321.00
BPP2	50	0.05	172004.05	2960.86	167253.25	174446.50
BPP2	50	0.30	143855.55	12667.98	126874.75	158713.25
BPP2	10	0.05	177969.70	26034.03	143774.50	199838.50
BPP2	10	0.30	146515.70	11286.18	129393.75	157343.25

3.2 Experimental Results

BPP1 strongly favours $p = 10, e = 0.30$, while BPP2 favours $p = 50, e = 0.30$. The common result is that $e = 0.30$ outperforms $e = 0.05$ for both population sizes and both problems. Fitness scales differ substantially between the problems, so values are compared only within each problem.

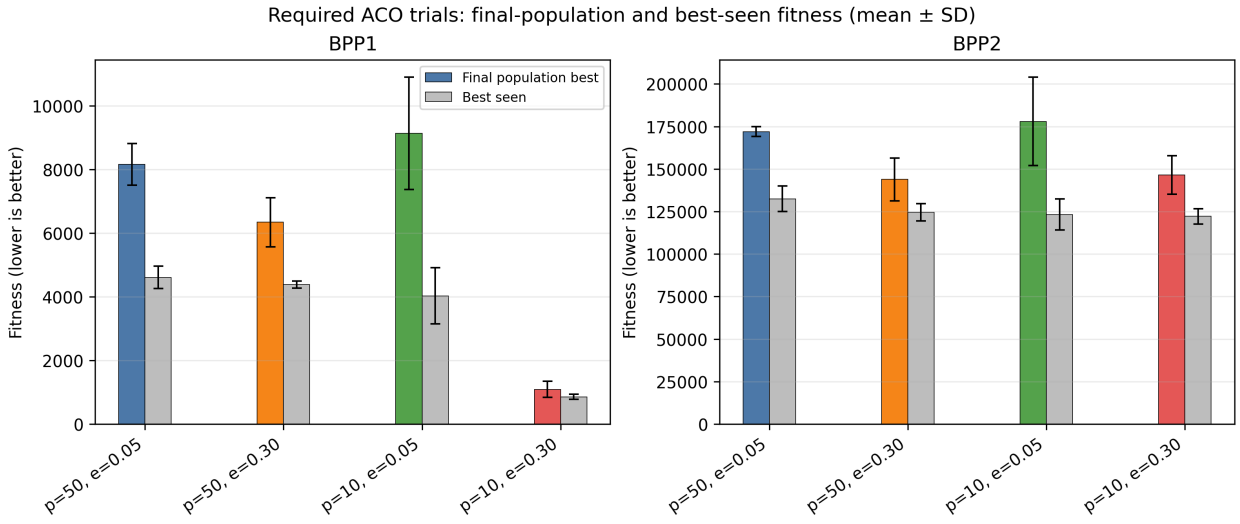


Figure 1: Mean final-population and best-seen fitness across the required trials; error bars show one standard deviation.

4 Discussion

4.1 Answer to Question 1

The best setting is problem-dependent. BPP1 is best with $p = 10, e = 0.30$, giving mean final fitness 1091.60. BPP2 is best with $p = 50, e = 0.30$, giving mean final

fitness 143855.55. The shared feature is high evaporation, so $e = 0.30$ is the strongest general choice in these experiments, while p should depend on the instance.

4.2 Answer to Question 2

At $e = 0.05$, 95% of pheromone is retained after each update. Since the initial table is random, early decisions can retain an accidental bias for too long. At $e = 0.30$, only 70% is retained, allowing later populations to replace misleading decisions. This provides a plausible explanation for the consistent advantage of high evaporation.

The size of the effect supports this interpretation. Increasing e from 0.05 to 0.30 reduces mean final fitness by 22.28% for $p = 50$ on BPP1 and by 88.06% for $p = 10$. On BPP2, the corresponding improvements are 16.37% and 17.67%. The especially large BPP1 improvement for $p = 10$ suggests that frequent updates and rapid pheromone replacement work particularly well for that instance.

Population size changes the number of updates available under 10,000 evaluations. $p = 10$ gives approximately 1,000 updates, while $p = 50$ gives approximately 200. BPP1 appears to benefit from rapid adaptation, whereas BPP2 benefits slightly from the broader sampling provided by 50 ants per update. These are tentative explanations because each setting has only five trials and the algorithm does not retain the global best route in its reported final population.

4.3 Answer to Question 3

A larger population samples more complete assignments per update and makes each pheromone update less dependent on one route, but it reduces the number of updates. A smaller population updates more frequently and can adapt quickly, but its updates are noisier. This explains why $p = 10$ is highly effective on BPP1 but not quite best on BPP2.

Low evaporation creates long-term memory and stronger exploitation, but can preserve poor early preferences. High evaporation increases adaptation and exploration, which improved both problems here. The figure also shows that the mean best-seen fitness is lower than the mean final-population fitness for most settings. This is expected because the required implementation reports the best ant in the final population and does not preserve an elite global solution. The difference is particularly important when interpreting convergence: finding a good route earlier does not guarantee that it remains in the final population. Excessive evaporation could eventually remove useful information too quickly, so the result supports $e = 0.30$ for these instances rather than all possible problems.

4.4 Answer to Question 4

A hybrid ACO with local search is the strongest candidate for improvement because a local move or swap can directly reduce the load range after an ant constructs a

route. Levine and Ducatelle found that local search strengthened ACO on some packing classes [6]. A grouping GA could also preserve useful item groups, while PSO would require a discrete assignment model and repair mechanism. SA or tabu search could provide effective local improvement.

However, the reviewed studies use different problem definitions and budgets. It is therefore not possible to claim that one alternative would definitely outperform this implementation without implementing and testing it under the same 10,000-evaluation limit.

5 Further Experiments

An additional intermediate setting, $p = 25, e = 0.15$, was tested using five independent trials on each problem and the same 10,000-evaluation limit.

Table 3: Additional $p = 25, e = 0.15$ experiment.

Problem	Mean	SD	Best	Worst
BPP1	8105.20	1586.77	5685.00	9595.00
BPP2	161330.45	6482.13	153290.50	167263.75

The intermediate setting did not beat the best required setting on either problem. Its BPP1 mean was much worse than $p = 10, e = 0.30$, and its BPP2 mean was worse than both $e = 0.30$ settings. It did improve on the low-evaporation BPP2 settings, supporting the conclusion that evaporation rate was more influential than an intermediate population size. Since only five additional trials were used, this is supporting evidence rather than a complete parameter search.

6 Conclusions

The required ACO experiments show that parameter choice and problem structure interact. BPP1 performed best with $p = 10, e = 0.30$, while BPP2 performed best with $p = 50, e = 0.30$. High evaporation was consistently beneficial, suggesting that rapid removal of random early pheromone bias improved exploration. The additional $p = 25, e = 0.15$ experiment did not improve the required results.

The literature suggests that local search, grouping-aware genetic operators, or discrete swarm operators could improve the algorithm, but such comparisons would need separate implementations and equal evaluation budgets. Overall, the basic ACO provides a transparent method for studying fixed-bin load balancing, and the results demonstrate that no single population size should be assumed to be optimal across different problem structures.

References

[1] ALVIM, A. C. F., RIBEIRO, C. C., GLOVER, F., AND ALOISE, D. J. A hybrid improvement heuristic for the one-dimensional bin packing problem. *Journal of Heuristics 10*, 2 (2004), 205–229.

[2] CHAN, T. S., AU, K. C., CHAN, L. Y., AND LAU, T. L. Using genetic algorithms to solve quality-related bin packing problem. *Robotics and Computer-Integrated Manufacturing 23*, 1 (2007), 71–81.

[3] DORIGO, M., DI CARO, G., AND GAMBARDELLA, L. M. Ant algorithms for discrete optimization. *Artificial Life 5*, 2 (1999), 137–172.

[4] DORIGO, M., MANIEZZO, V., AND COLORNI, A. Ant system: Optimization by a colony of cooperating agents. *IEEE Transactions on Systems, Man, and Cybernetics, Part B: Cybernetics 26*, 1 (1996), 29–41.

[5] HEMMELMAYR, V., HIRSCH, R. F., AND SCHMID, V. Variable neighbourhood search for the variable sized bin packing problem. *Computers and Operations Research 39*, 5 (2012), 1097–1108.

[6] LEVINE, J., AND DUCATELLE, F. Ant colony optimisation and local search for bin-packing and cutting stock problems. *Journal of the Operational Research Society 55*, 7 (2004), 705–716.

[7] LIU, D. S., TAN, K. C., HUANG, S. Y., GOH, C. K., AND HO, W. K. On solving multiobjective bin packing problems using evolutionary particle swarm optimization. *European Journal of Operational Research 190*, 2 (2008), 357–382.

[8] MUNIEN, C., AND EZUGWU, A. E. Metaheuristic algorithms for one-dimensional bin-packing problems: A survey of recent advances and applications. *Journal of Intelligent Systems 30*, 1 (2021), 636–663.

[9] RAO, R. L., AND IYENGAR, S. S. Bin-packing by simulated annealing. *Computers and Mathematics with Applications 27*, 5 (1994), 71–82.

[10] STUTZLE, T., AND HOOS, H. H. Max–min ant system. *Future Generation Computer Systems 16*, 8 (2000), 889–914.

[11] YAKAWA, T., IIMA, H., AND SANNOMIYA, N. Proposition of genetic algorithm for bin packing problems. *Transactions of the Society of Instrument and Control Engineers 41*, 3 (2005), 274–282.